ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ

Проект «Воздушный шар»

Игровая web-платформа с серверной случайностью, турнирной системой, лотереей и ежедневными наградами.

Актуализировано: 13 сентября 2026 г.

1. Назначение и состав системы

«Воздушный шар» — прототип crash-игры. Игрок выбирает тему и ставку, шар поднимается по уровням, а выплату необходимо зафиксировать до взрыва. Backend является источником истины для результата, денег, очков, билетов и наград.

Возможности

- регистрация, вход, серверные сессии и роли USER/ADMIN;
- красная и зелёная темы, ставки, бустеры, cashout и автоматическая выплата на вершине;
- история, proof честности, рейтинг, турнир, пазл, календарь входа и лотерея;
- административная настройка параметров и аудит действий.

Стек

Слой	Технологии	Назначение
Frontend	React 18, TypeScript, Vite, Router, TanStack Query, Zustand	SPA, маршрутизация, server cache и UI state
Backend	Java 21, Spring Boot 3.3.5, MVC, Validation, JPA	REST API, игровая логика и транзакции
Данные	H2 file database	Фактическое постоянное хранилище
Запуск	Docker Compose, nginx; PostgreSQL и Redis	nginx проксирует API; PostgreSQL/Redis пока не используются бизнес-логикой

Адреса и доступ

- Приложение: http://localhost:3000; API: http://localhost:8080/api/v1.
- Swagger UI: http://localhost:3000/swagger-ui/index.html.
- Игрок demo / demo123; администратор admin / admin123.
- Новый аккаунт получает стартовый баланс 1000 бонусов.

2. Архитектура

Система разделена на браузерный SPA, nginx, Spring Boot API и постоянное хранилище. Клиент отправляет команды и отображает ответ, но не определяет исход раунда.

Компонент	Связь	Компонент назначения
React SPA	HTTP: /api/**	nginx reverse proxy
nginx	HTTP внутри Compose	Spring Boot :8080
Spring Boot / JPA	JDBC	H2 file в volume backend-data
Compose	отдельные сервисы	PostgreSQL :5432 и Redis :6379 — инфраструктурный задел

Backend-модули

Пакет	Ответственность
api	REST-контроллеры auth, game, rewards, lottery и admin
application	GameService, LotteryService, LeaderboardService
auth	аккаунты, хэши паролей, сессии, роли, аудит
daily	дневной пазл и 7-дневный календарь входа
rounds / tournament / config	персистентность раундов, турнира и актуальной конфигурации

Важно о конфигурации

GameConfigEntity хранит одну актуальную запись на каждую тему. Поле version увеличивается при публикации и попадает в commitment, однако отдельная история конфигурационных записей не ведётся.

Поток раунда

- POST /rounds списывает ставку, создаёт seed, commitment и запись раунда.
- Планировщик сервера поднимает шар ровно на один уровень каждые 1,5 секунды; уровень не вычисляется непрерывно «по времени».
- Frontend опрашивает GET /rounds/{id} каждые 500 мс и анимирует переход.
- Cashout фиксирует выплату, а полёт продолжает исполняться; состояние остаётся CASHED_OUT.
- Финал сохраняет историю, очки и награду пазла.

3. Игровая модель и обоснование

Шансы

```
hazard(1) = 0
hazard(L) = min(maxHazard, baseHazard × hazardGrowth^(L-1)), L ≥ 2
survival(L) = Π(1 - hazard(i)), i = 1..L
multiplier(L) = rtp / survival(L)
```

Формула hazard применяется только при выключенной ручной таблице hazardOverride. При полной валидной таблице сервер использует заданную вероятность каждого уровня; первый уровень всё равно обязан иметь вероятность 0.

Почему multiplier = rtp / survival

Вероятность дожить до уровня L равна survival(L). Для ставки bet сервер рассчитывает payout = max(1, round(bet × multiplier(L))): произведение округляется до ближайшего целого, минимальная выплата равна 1 бонусу. До округления математический возврат равен survival(L) × bet × rtp / survival(L) = bet × rtp. Поэтому ожидаемый возврат одинаков на любом уровне, хотя риск и величина единичной выплаты растут.

Параметры тем

Тема	Уровни	baseHazard	growth	maxHazard	RTP
Красная	12	0,15	1,14	0,60	0,96
Зелёная	9	0,10	1,25	0,60	0,96

Пример красной формульной темы

Уровень	Шанс взрыва	Шанс долететь	Коэффициент, примерно
1	0%	100%	×1,01
2	17,1%	82,9%	×1,16
3	19,5%	66,7%	×1,44
4	22,2%	51,9%	×1,85
5	25,3%	38,8%	×2,48
8	37,5%	11,6%	×8,31
12	60,0%	0,6%	×159,75

Первое значение сервер поднимает до ×1,01, поскольку лестница обязана возрастать минимум на 0,01. Значения округлены; точная таблица строится сервером из активной конфигурации. Ручные hazardOverride и ladderOverride меняют эту таблицу.

Очки

Базовые очки: points = достигнутые уровни × pointsPerLevel. При cashout добавляется pointsCashoutBonus. При активации бустера добавляется pointsBoosterBonus × boosterMultiplier. После cashout базовые очки продолжают начисляться до конца фактического полёта, но бустер больше не активируется.

4. Бустеры, честность и состояния

Бустеры

Шанс появления зависит от множителя B: min(1, 0,5/(B-1)). Для ×2 это 50%, для ×3 — 25%, для ×4 — около 16,7%. Поэтому бустер выпадает не в каждом раунде. Его уровень выбирается по весам не раньше третьего уровня.

Provably fair

Перед стартом публикуется commitment = SHA-256(seed:roundId:configVersion). На каждом рискованном уровне сервер сравнивает HMAC-SHA256(seed, roundId:level) с hazard уровня. После завершения GET /rounds/{id}/proof раскрывает seed, источник hazard и popLevel. Режима с пользовательским или фиксированным seed нет.

Фактические состояния

Состояние	Смысл
CREATED	Зарезервировано в модели, сервером не присваивается; новый раунд сразу создаётся в STARTED
STARTED	Раунд запущен и ещё не завершён
CASHED_OUT	Выплата зафиксирована; полёт может продолжаться, состояние после взрыва не меняется на CRASHED
CRASHED	Шар взорвался до cashout
SETTLED	Шар достиг вершины без cashout, выполнена автоматическая выплата
REFUNDED	Сервис перезапустился до cashout; ставка возвращена

Ограничения игрового сценария

- Cashout доступен только со второго уровня.
- «Играть снова» на результате сразу запускает новый раунд с прежней темой и ставкой; это не просто возврат на /play.
- Живой рейтинг на игровом экране — топ-5, обновляется раз в 15 секунд.
- После завершения текущего турнира новый автоматически не создаётся.
- Персональные предложения лотереи находятся в памяти GameService и теряются при перезапуске backend.

5. Административная настройка параметров

Администратор открывает /admin, выбирает тему и публикует конфигурацию через PUT /admin/config. Сервер валидирует значения, увеличивает version и сохраняет одну актуальную запись темы в H2. Изменения применяются только к новым раундам; уже созданный раунд продолжает использовать собственный снимок конфигурации.

Параметр	Допустимое значение / правило	Влияние
levels	фиксировано для темы: RED 12, GREEN 9	Длина полёта и таблиц
bets	ровно 4 ставки, каждая ≥ 1	Стоимость старта и база выплаты
boosterMultiplier	1–100	Множитель выплаты/очков
boosterWeights	≥ 0; уровни 1–2 игнорируются; хотя бы один вес с уровня 3 должен быть > 0	Распределение уровня бустера; иначе 400
points*	≥ 0; общий максимум помещается в int	Турнирные очки
baseHazard	0,001–0,5	Базовый риск формулы
hazardGrowth	1,0–2,0	Темп роста риска
maxHazard	от baseHazard до 0,95	Потолок риска
hazardOverride	ровно levels значений 0–0,95; уровень 1 = 0	Полная ручная таблица рисков
rtp	0,5–1,0	Ожидаемый возврат и лестница
ladderOverride	полная возрастающая таблица, значения от 1,01	Ручные коэффициенты
minWinAmount	≥ 0	Порог предложения лотереи
popupTimeout	3–60 секунд	Время доступности upsell

Как настройки меняют игру

- Рост baseHazard/hazardGrowth или ручных hazard увеличивает ранние взрывы; автоматически рассчитанные коэффициенты растут быстрее.
- Рост RTP линейно повышает рассчитанные коэффициенты и ожидаемый возврат.
- ladderOverride отделяет коэффициенты от автоматического расчёта; экономику надо проверять симуляцией.
- Изменение очков влияет на новые раунды и турнирный рейтинг.

Сброс

POST /admin/config/reset возвращает тему к серверным значениям по умолчанию и публикует следующую версию.

6. Пользователи и награды

Аккаунты и сессии

Пароль хранится в виде PBKDF2-хэша. Bearer-токен непрозрачный; в sessions сохраняется только SHA-256 токена. Срок сессии — 30 дней, продление не чаще одного раза в час. Cookie-аутентификация не используется, поэтому CSRF не является основной угрозой; для production важнее строгий CORS allow-list, TLS, защита токена и rate limiting.

Дневной пазл

- За завершённый раунд выдаётся случайный отсутствующий фрагмент из 6.
- Дата определяется сервером в Europe/Moscow.
- После 6/6 POST /daily-bonus/claim атомарно начисляет 10–100 бонусов; повтор не начисляет снова.

7-дневный календарь входа

День	Награда
1	10 монет
2	20 монет
3	30 монет
4	2 фрагмента пазла
5	40 монет
6	2 фрагмента пазла
7	1 лотерейный билет

Повторный claim в те же московские сутки возвращает 409. Последовательный вход продвигает цикл; пропуск хотя бы дня сбрасывает его на день 1.

Лотерея

Покупка списывает бонусы и добавляет билеты. Стирание списывает один билет и начисляет серверный приз в одной транзакции. Upsell появляется раз в три подходящих выигрыша, но незавершённое предложение не переживает перезапуск backend.

7. REST API

Базовый путь — /api/v1. Защищённые операции требуют Authorization: Bearer TOKEN.

Метод	Маршрут	Назначение
POST	/auth/register, /login, /logout	Аккаунты и сессии
POST	/auth/topup	+500 раз в московский день
GET	/me	Профиль и экономика
GET/POST	/login-reward, /login-reward/claim	Календарь входа
GET/POST	/daily-bonus, /daily-bonus/claim	Пазл и дневной билет
GET	/game/config, /rules	Конфигурация и правила
POST	/rounds	Старт раунда
GET/POST	/rounds/{id}, /rounds/{id}/cashout	Состояние и cashout владельца
GET	/rounds/{id}/proof	Проверка честности
GET	/rounds/history, /rounds/mine, /rounds/crashes	История
GET	/leaderboard, /tournament	Рейтинг и турнир
GET/POST	/lottery, /buy, /scratch, /offer/**	Лотерея
GET/PUT/POST	/admin/config, /admin/config/reset	Настройка ADMIN
GET/POST/PUT	/admin/users/**, /admin/audit	Пользователи ADMIN

HTTP-ошибки

400 — валидация; 401 — нет сессии; 403 — недостаточно роли; 404 — нет ресурса или скрыт чужой roundId; 409 — конфликт состояния; 500 — непредвиденная ошибка.

8. Примеры проверки API

```
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login -H "Content-Type: application/json" -d
'{"username":"demo","password":"demo123"}' | jq -r .token)

curl -X POST http://localhost:8080/api/v1/rounds -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -H "Idempotency-Key:
smoke-001" -d '{"theme":"RED","betOptionId":"red-x2"}'
```

Cashout и proof

```
curl -X POST http://localhost:8080/api/v1/rounds/ROUND_ID/cashout -H "Authorization: Bearer $TOKEN"

curl http://localhost:8080/api/v1/rounds/ROUND_ID/proof -H "Authorization: Bearer $TOKEN"
```

Сценарий 5: изменение параметра администратором

```
ADMIN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login -H "Content-Type: application/json" -d
'{"username":"admin","password":"admin123"}' | jq -r .token)

curl -X PUT "http://localhost:8080/api/v1/admin/config?theme=RED" -H "Authorization: Bearer $ADMIN" -H "Content-Type: application/json" -d
'{"pointsPerLevel":25}'

# Следующий новый раунд использует 25 очков за уровень.
# Возврат значения:
curl -X PUT "http://localhost:8080/api/v1/admin/config?theme=RED" -H "Authorization: Bearer $ADMIN" -H "Content-Type: application/json" -d
'{"pointsPerLevel":10}'
```

Ручные шансы и возврат к формуле

```
curl -X PUT "http://localhost:8080/api/v1/admin/config?theme=GREEN" -H "Authorization: Bearer $ADMIN" -H "Content-Type: application/json" -d
'{"hazardOverride":[0,0.10,0.12,0.15,0.20,0.27,0.36,0.48,0.60]}'

# Отключить ручную таблицу и снова использовать формулу:
curl -X PUT "http://localhost:8080/api/v1/admin/config?theme=GREEN" -H "Authorization: Bearer $ADMIN" -H "Content-Type: application/json" -d
'{"hazardOverride":[]}'
```

PUT поддерживает частичное обновление: неуказанные поля сохраняются. Точные модели запросов приведены в openapi.yaml и Swagger UI.

9. Данные и безопасность

Сущность	Содержимое
AccountEntity	баланс, очки, роли, пазл, билеты, дни наград
SessionEntity	SHA-256 токена, lastUsedAt, expiresAt
RoundEntity	ставка, снимок параметров, seed, commitment, payout, state
GameConfigEntity	одна актуальная конфигурация каждой темы
DailyBonusEntity	пазл и билет на account/date
TournamentEntity	один текущий турнир и призы
AuditEntity	административные действия

Транзакционные гарантии

- Баланс, билеты и награды меняются под pessimistic write lock аккаунта.
- daily_bonuses имеет уникальность account_id + bonus_date.
- Idempotency-Key старта исключает повторное списание при повторе запроса.
- Чужой roundId маскируется ответом 404.

Хранение

Рабочая база — jdbc:h2:file:./data/balloon в Docker volume backend-data. PostgreSQL и Redis подняты, но не подключены к бизнес-данным. Hibernate ddl-auto=update меняет схему автоматически; для production нужны PostgreSQL и Flyway/Liquibase.

Резервирование

- Перед backup остановить backend или получить согласованный snapshot.
- Копировать backend-data; удаление этого volume удаляет рабочую базу.
- Регулярно проверять восстановление на отдельном окружении.

10. Frontend и интерфейс

URL	Экран	Доступ
/	Landing и вход	публичный
/play	ставка и календарь наград	любой вошедший; API защищает данные
/round/:id	полёт	любой вошедший; чужой roundId даёт 404 от API
/round/:id/result	результат и replay	любой вошедший; владелец проверяется API
/leaderboard	турнир	USER
/lottery	билеты	USER
/history	личная/общая история	USER
/rules, /profile	правила и профиль	USER
/admin	настройки и пользователи	ADMIN

Синхронизация

TanStack Query хранит server cache. После мутаций инвалидируются me, daily-bonus, login-reward и lottery. Живой топ-5 обновляется каждые 15 секунд, состояние активного раунда — каждые 500 мс. Лента «Прошлые игры» теперь показывается на экране выбора ставки, во время полёта и в разделе истории.

Дизайн

Используются адаптивные сетки, полупрозрачная шапка, белые контентные секции, 3D-ассеты шаров, монет, пазлов, медалей, билетов и коробок. Денежные значения показывают иконку монеты. Фоновый звук птиц запускается во всём приложении, а выбор шара сопровождается звуком капли.

Доступность

- Кнопки имеют disabled и aria-состояния.
- prefers-reduced-motion ограничивает декоративное движение.
- Ошибка backend не подменяется локальной транзакцией, если VITE_DEMO_MODE не включён явно.

11. Сборка и эксплуатация

Docker Compose

```
docker compose -f infra/docker-compose.yml up --build -d
docker compose -f infra/docker-compose.yml ps
docker compose -f infra/docker-compose.yml logs -f backend frontend
```

Сервис	Порт	Роль
frontend	3000 → 80	nginx + SPA + proxy
backend	8080	Spring Boot API
postgres	5432	инфраструктурный задел
redis	6379	инфраструктурный задел

Локальная разработка

```
cd backend
gradle bootRun

cd frontend
npm install
npm run dev
```

Безопасная очистка Docker

```
docker builder prune -af
docker image prune -f
docker container prune -f
```

Не выполнять docker compose down -v и docker volume prune без резервной копии: backend-data содержит H2. Начинать очистку следует с build cache и неиспользуемых образов.

Smoke-check

- GET /actuator/health; вход demo; старт и cashout; изменение баланса и истории.
- Получение ежедневной награды и 409 при повторе.
- Лотерея, рейтинг, /proof и доступ ADMIN.

12. Тестирование, ограничения и развитие

Фактические backend-тесты

Тест	Что проверяет
ModelsOddsTest	вероятности, лестницу, миграцию дефолтов и RTP
ModelsConfigTest	число уровней, рост лестницы и бустер; не серверную валидацию
GameServiceSecurityIntegrationTest	авторизацию и владение раундом
RoundPersistenceIntegrationTest	сохранение и восстановление состояний
AccountServiceSecurityIntegrationTest	сессии, роли и блокировки
DailyBonusServiceIntegrationTest	пазл и идемпотентность
Lottery*Test	лотерею и безопасность

Команды

```
cd backend && gradle test
cd frontend && npm run build
docker compose -f infra/docker-compose.yml build backend frontend
```

Известные ограничения

- H2 и ddl-auto=update недостаточны для production.
- Нет истории конфигураций, фиксированного seed и автоматического следующего турнира.
- Upsell-предложения не сохраняются между перезапусками.
- Realtime построен на polling, а не WebSocket.

Рекомендации

- Перейти на PostgreSQL + Flyway, добавить backup/restore-test.
- Добавить интеграционные тесты LoginRewardService: граница МСК, конкурирующие claim и пропуск дня.
- Синхронизировать openapi.yaml с login-reward и hazardOverride.
- Убрать демо-пароли из production, включить TLS, CORS allow-list и rate limiting.
- Любое изменение hazard/ladder сопровождать симуляцией экономики и проверкой RTP.

Документ сформирован по фактической реализации проекта и актуальному состоянию master на хосте. Его следует обновлять вместе с API, схемой данных и игровой экономикой.